

IA: MACHINE LEARNING FOUNDATIONS

SESSION 8 – Tree-based Methods

Adrián Sánchez Morales

AGENDA

1. Decision Trees
   - 1.1. Regression
   - 1.2. Classification
   - 1.3. Pros and Cons
   - 1.4. Trees and KNN

2. Bagging
   - 2.1. Introduction
   - 2.2. Random Forests

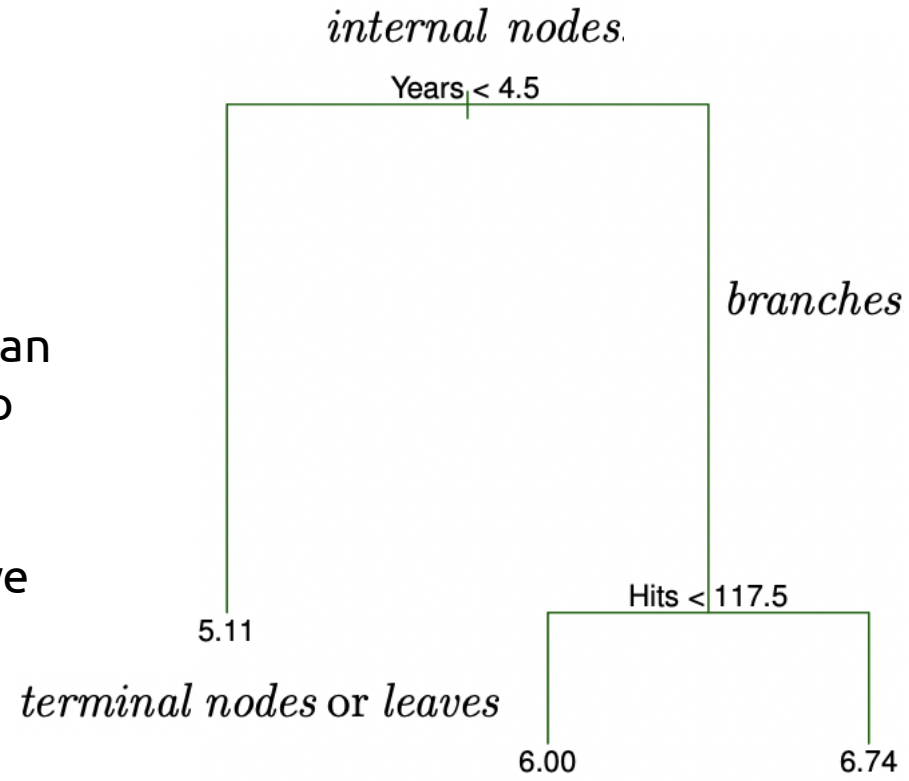
3. Boosting
   - 3.1. Introduction
   - 3.2. AdaBoost
   - 3.3. XGBoost
   - 3.4. LightGBM



1. DECISION TREES

1.1. REGRESSION

- Tree-based methods involve stratifying or segmenting the predictor space into simple regions.
- Then, to predict a given observation, we just simply use the mean or the mode value for the training observations in the region to which it belongs.
- Trees are simple and good at interpretation but not competitive with other methods.
- Multiples trees together improve this scenario.
- Example in **Hitters data set**: predict ***Salary*** from ***Years*** and ***Hits***.



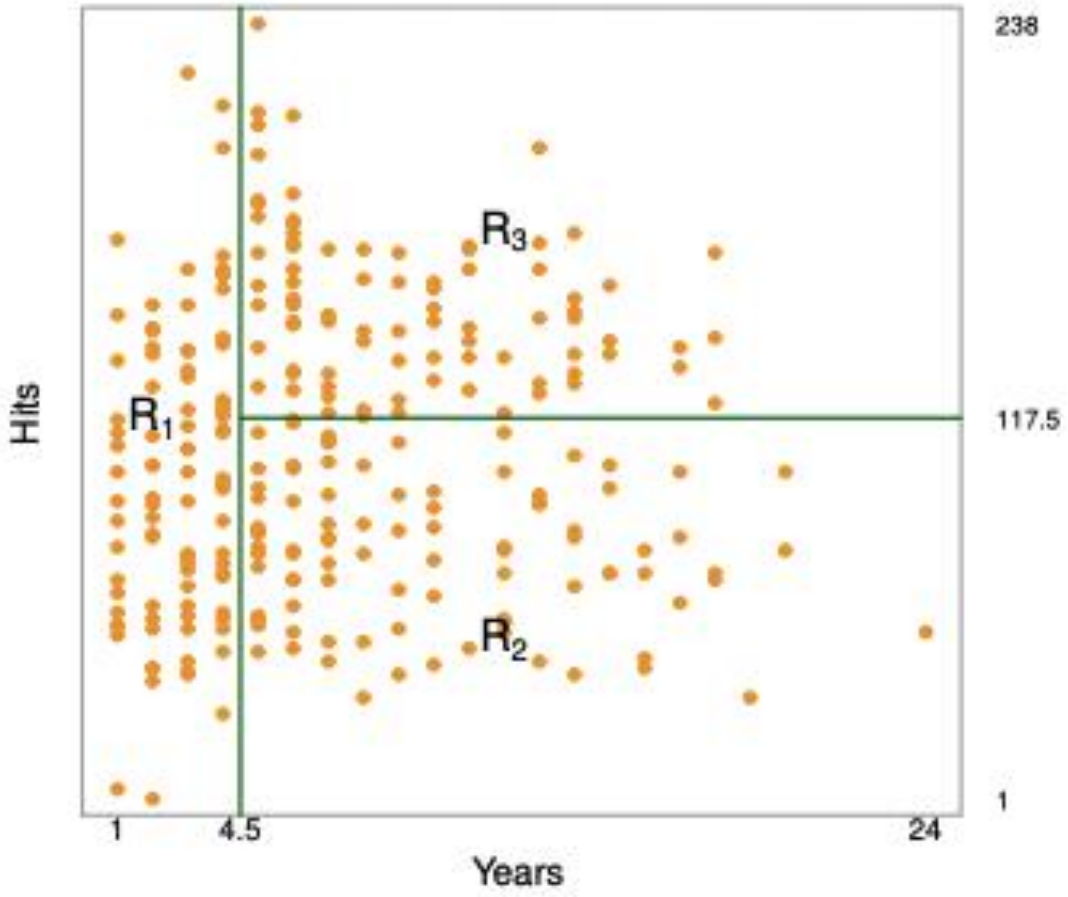
$$R_1 = \{X \mid \texttt{Years<4.5}\}$$

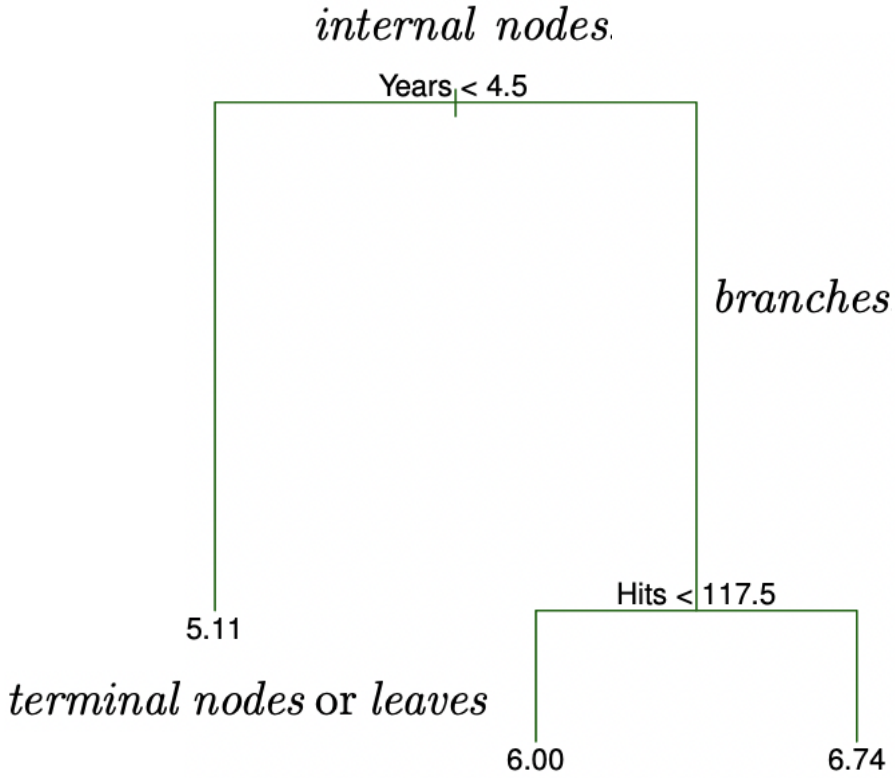
$$R_2 = \{X \mid \texttt{Years>=4.5}, \texttt{Hits<117.5}\}$$

$$R_3 = \{X \mid \texttt{Years>=4.5}, \texttt{Hits>=117.5}\}$$

1. DECISION TREES

1.1. REGRESSION





$$R_1 = \{X \mid \texttt{Years<4.5}\}$$

$$R_2 = \{X \mid \texttt{Years>=4.5}, \texttt{Hits<117.5}\}$$

$$R_3 = \{X \mid \texttt{Years>=4.5}, \texttt{Hits>=117.5}\}$$

1. DECISION TREES

1.1. REGRESSION

- How is the process of building a regression tree?
  1. Divide the predictors space into *J* distinct and no overlapping regions $R_1, R_2, \ldots, R_J$ by using the set of possible valu $X_1, X_2, \ldots, X_p$
  2. For every observation that falls into a specific region we simply make the same prediction

- Regions could have any shape, but rectangles are used for simplicity and interpretation.

- How do we construct the regions?

$$\sum_{j=1}^{J} \sum_{i \in R_j} (y_i - \hat{y}_{R_j})^2,$$

Recursive Binary Splitting

Top-down – it starts at the top of the tree (all observations belong to a single region) and splits the predictor space creating two new branches

Greedy – at each step, the best split is made rather than looking ahead and picking another split that may lead to a better tree in future steps

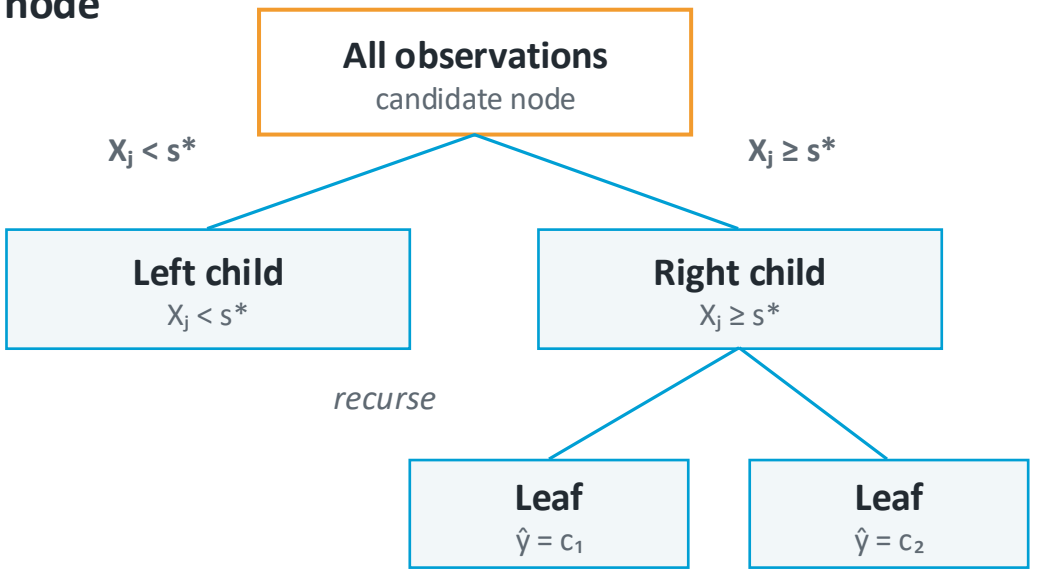
1. DECISION TREES

1.1. REGRESSION

Recursive Binary Splitting builds a tree one greedy split at a time

1. **Explore candidate cuts**
   For every feature j and threshold s, test a binary partition.
2. **Score each candidate**
   Calculate the weighted impurity of the two child nodes.
3. **Keep the best local split**
   Choose (j*, s*) with the largest immediate impurity reduction.
4. **Partition and recurse**
   Send observations left or right, then repeat inside each child.
5. **Stop and control complexity**
   End when a stopping rule is met; prune to reduce overfitting.

The same rule creates two branches at every node



Score Gini / entropy (classification) · RSS / MSE (regression)

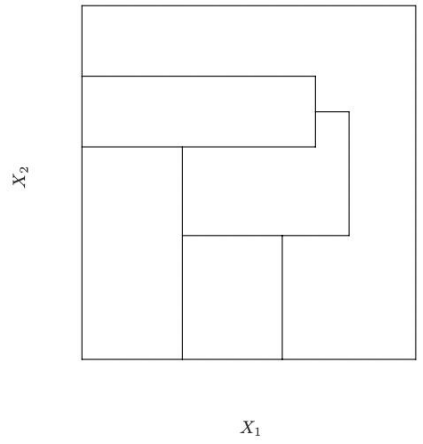
Stop max depth · min samples · min gain

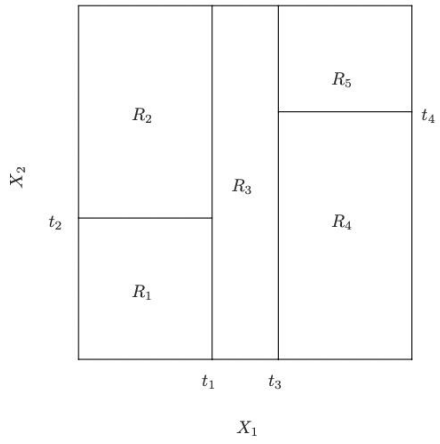
1. DECISION TREES

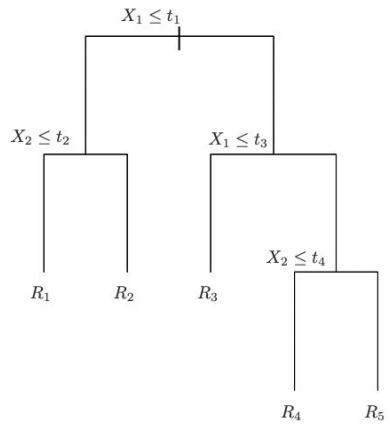
1.1. REGRESSION

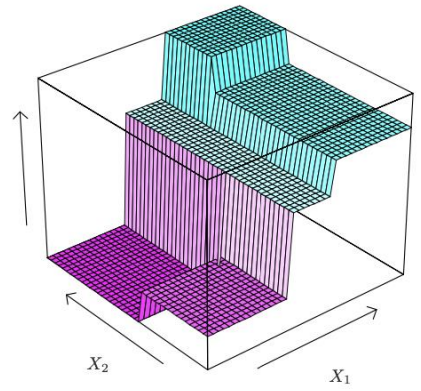
- Five-region example:
  - *Top left: A partition of two-dimensional feature space that could not result from recursive binary splitting.*
  - *Top right: The output of recursive binary splitting on a two-dimensional example.*
  - *Bottom left: A tree corresponding to the partition in the top right panel.*
  - *Bottom right: A perspective plot of the prediction surface corresponding to that tree.*

Recursive Binary Splitting









1. DECISION TREES

1.1. REGRESSION

- The previous process may produce very complex trees resulting in overfitting.
- On the other hand, a smaller tree with fewer splits (that is, fewer regions R1,...,RJ) might lead to lower variance and better interpretation at the cost of a little bias.
- There is a good strategy called ***Pruning**** that aims to create a large tree and then "prune" it back in order to obtain a subtree.

Building a Regression Tree

1. Use recursive binary splitting to grow a large tree on the training data, stopping only when each terminal node has fewer than some minimum number of observations.
2. Apply cost complexity pruning to the large tree in order to obtain a sequence of best subtrees, as a function of α.
3. Use K-fold cross-validation to choose α. That is, divide the training observations into K folds. For each $k = 1, \ldots, K$:
   (a) Repeat Steps 1 and 2 on all but the kth fold of the training data.
   (b) Evaluate the mean squared prediction error on the data in the left-out kth fold, as a function of α.
   Average the results for each value of α, and pick α to minimize the average error.
4. Return the subtree from Step 2 that corresponds to the chosen value of α.

*NOTE: Investigate about the mathematical concepts behind *cost complexity pruning*

1. DECISION TREES

1.1. REGRESSION

Cost Complexity Pruning

Choose a subtree T of the large tree T_0 that minimizes:

$$C_\alpha(T) = RSS(T) + \alpha \cdot |T|$$

RSS(T): sum of squared training errors (impurity)

|T|: number of terminal nodes (leaves)

$\alpha \geq 0$: penalty for each extra leaf

Balances training fit and tree size

What does α control?

α = 0: no complexity penalty.

Larger α favors smaller trees.

How is the subtree selected?

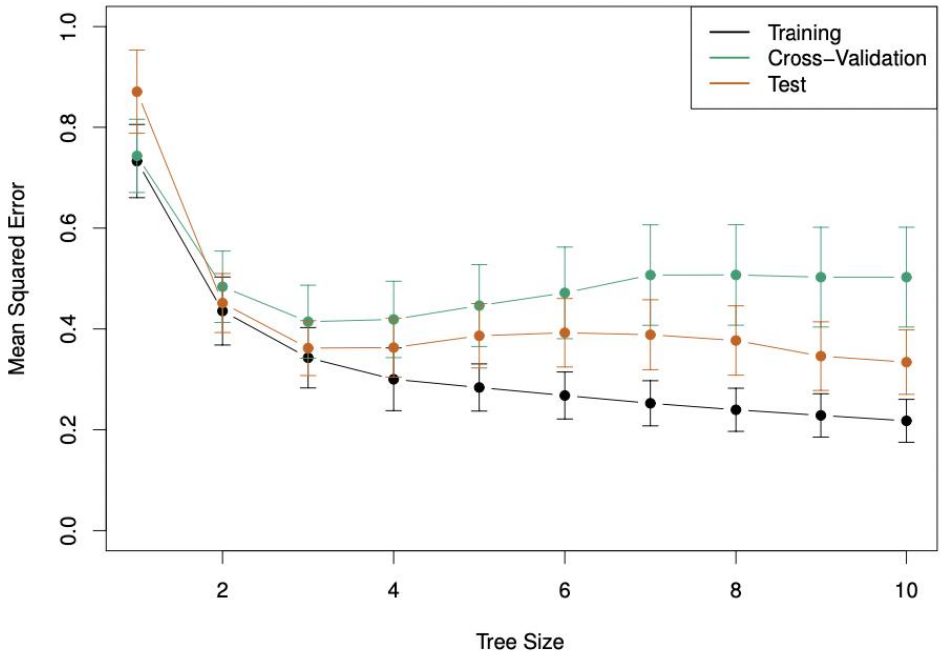
At each step, prune the branch that increases RSS the least per leaf removed.

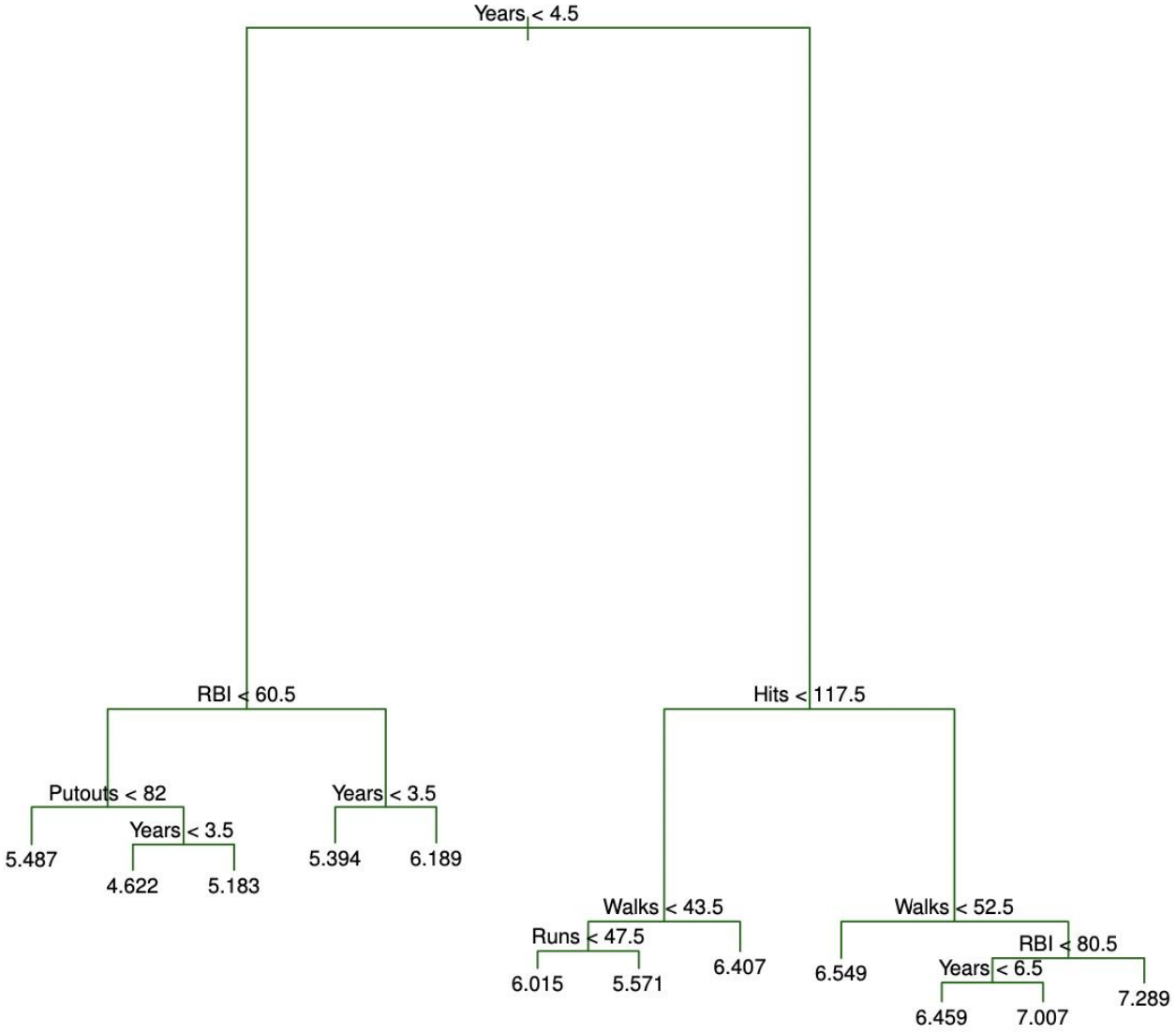
Use K-fold cross-validation to select α.

1. DECISION TREES

1.1. REGRESSION

Hitters Dataset





1. DECISION TREES

1.2. CLASSIFICATION

- A classification tree is very similar to a regression one, except that it is used to predict a qualitative response.
- A prediction is made by using the most commonly occurring class of the region.
- The task of creating a classification tree is similar, but the classification error rate is used rather than RSS.

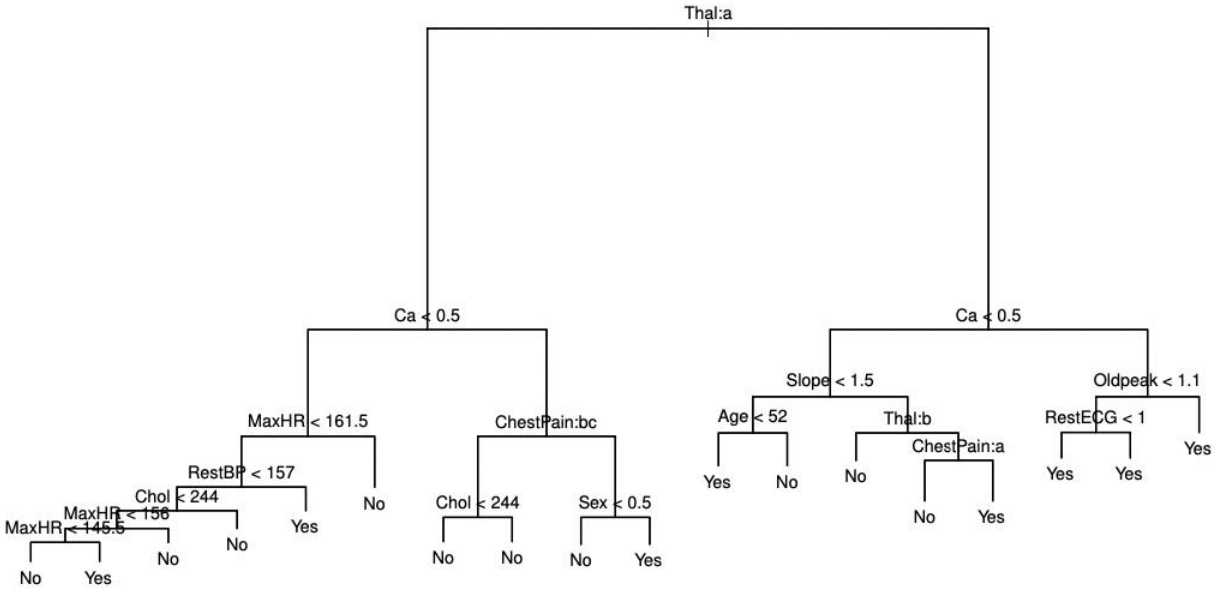
$$E = 1 - \max_k(\hat{p}_{mk})$$

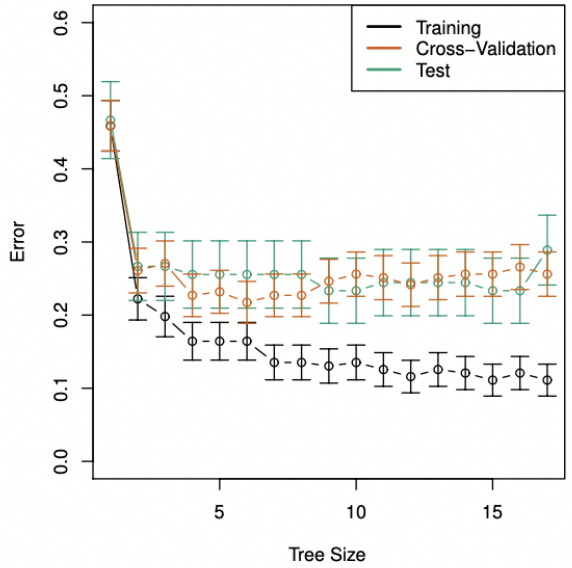
- In practice, ***Gini index*** and ***Entropy*** are typically used for the pruning process to evaluate the quality of a particular split.

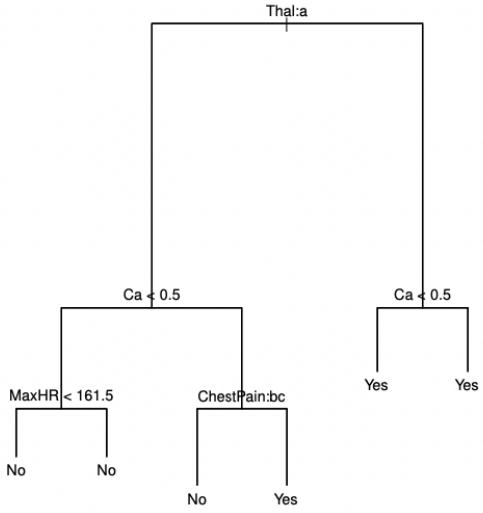
1. DECISION TREES

1.2. CLASSIFICATION

Heart Dataset







1. DECISION TREES

1.3. PROS AND CONS

▲ Trees are very easy to explain to people. In fact, they are even easier to explain than linear regression!
▲ Some people believe that decision trees are closer to human decision-making than do the regression and classification approaches seen in previous chapters.
▲ Trees can be displayed graphically, and are easily interpreted even by a non-expert (especially if they are small).
▲ Trees can easily handle qualitative predictors without the need to create dummy variables.

▼ Unfortunately, trees generally do not have the same level of predictive accuracy as some of the other regression and classification approaches.
▼ Additionally, trees can be very non-robust. In other words, a small change in the data can cause a large change in the final estimated tree.

1. DECISION TREES

1.4. TREES AND KNN

- Both are non-parametric. This means that the data distribution cannot be defined in a few parameters. In other words, Decision trees and KNN's don't have an assumption on the distribution of the data.
- Both can be used for regression and classification problems.
- Decision tree supports automatic feature interaction, whereas KNN doesn't.
- Decision trees can be faster. KNN tends to be slower with large datasets because it scans the whole dataset to predict as it doesn't generalize the data in advance.

AGENDA

1. Decision Trees
   - 1.1. Regression
   - 1.2. Classification
   - 1.3. Pros and Cons
   - 1.4. Trees and KNN

2. Bagging
   - 2.1. Introduction
   - 2.2. Random Forests

3. Boosting
   - 3.1. Introduction
   - 3.2. AdaBoost
   - 3.3. XGBoost
   - 3.4. LightGBM



2. BAGGING

2.1. INTRODUCTION

- An *ensemble* method is an approach that combines many simple “building block” (*weak learners*) models in order to obtain a single and potentially very powerful model.

- What was the idea behind Bootstrap?

2. BAGGING

2.1. INTRODUCTION

- An *ensemble* method is an approach that combines many simple "building block" (*weak learners*) models in order to obtain a single and potentially very powerful model.
- Now we use this methodology in a different context: improving the statistical learning method of decision trees.
- The trees seen so far suffer from high variance: if we randomly split a training dataset in two parts and train the tree, we will get quiet different results.
- *Bootstrap aggregation* (*bagging*) can be used to reduce variance:
    1. Taking repeated samples from the single training dataset (*B* different bootstrapped datasets)
    2. Training a model for each set (not pruned trees)
    3. Average the prediction

$$\hat{f}_{\text{bag}}(x) = \frac{1}{B}\sum_{b=1}^{B}\hat{f}^{*b}(x)$$

2. BAGGING

2.1. INTRODUCTION

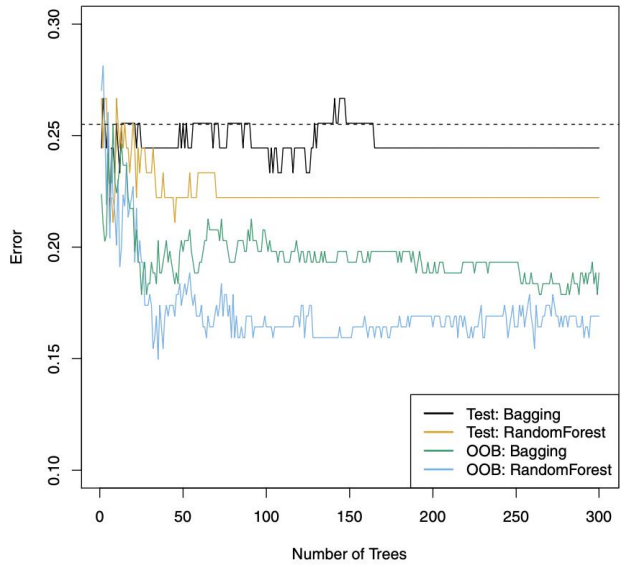
- Bagging can be used for many classification/regression methods, but it is especially powerful in decision trees.
- Bagging has been demonstrated to give impressive improvements in accuracy by combining hundreds or even thousands of trees into a single procedure.
- How do use bagging for classification?

2. BAGGING

2.1. INTRODUCTION

- Bagging can be used for many classification/regression methods, but it is especially powerful in decision trees.
- Bagging has been demonstrated to give impressive improvements in accuracy by combining hundreds or even thousands of trees into a single procedure.
- How do use bagging for classification? → *Voting*

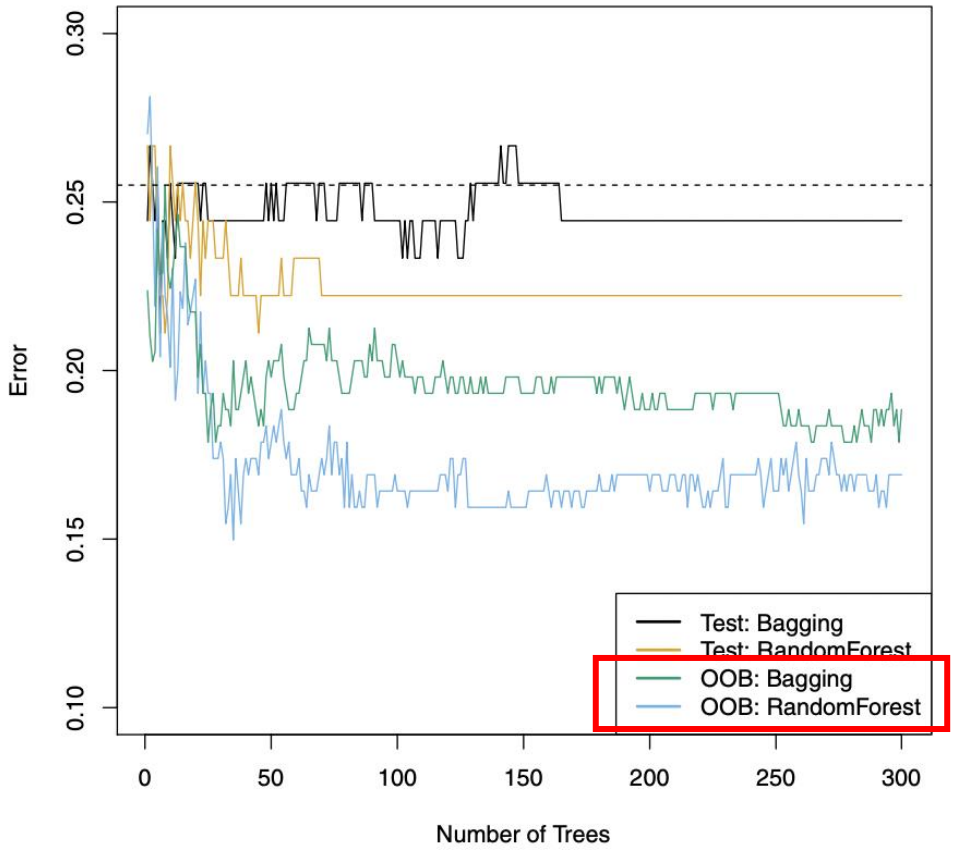
  - Number of trees *B* is not critical
  - Large *B* will not lead overfitting
  - In practice, use a value B sufficiently large that the error has settled down



Single Tree

2. BAGGING

2.1. INTRODUCTION



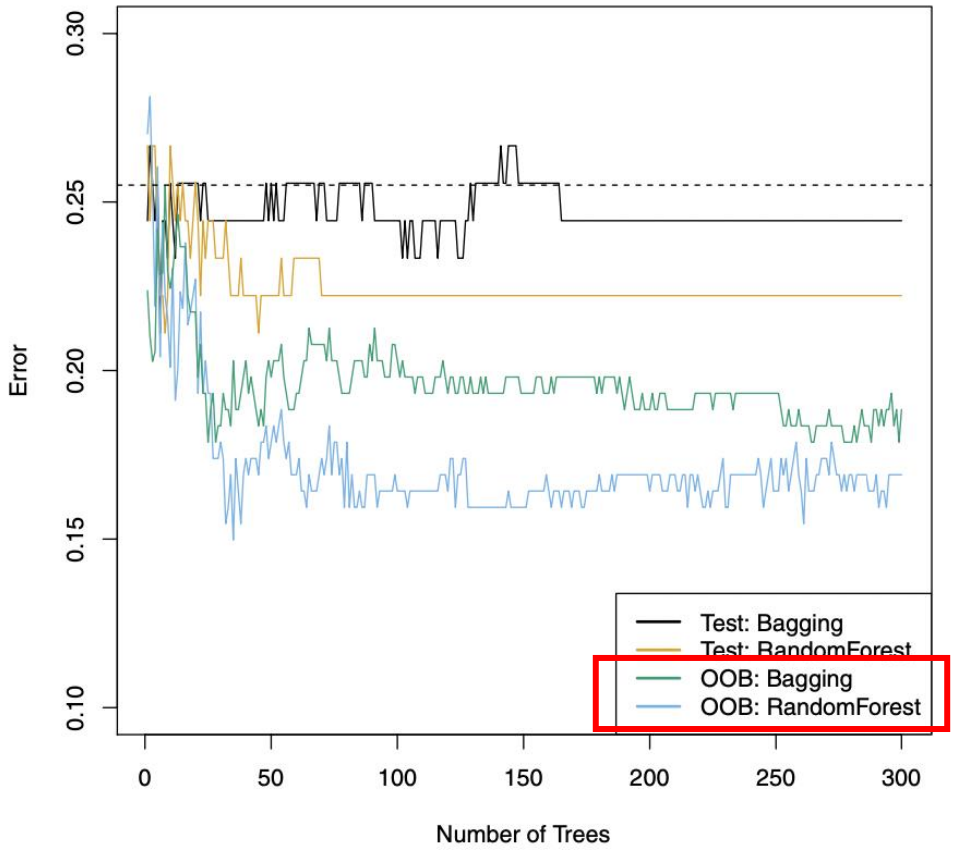
OOB?

DISCUSSION

Interpretability & Variable Importance in Bagging

2. BAGGING

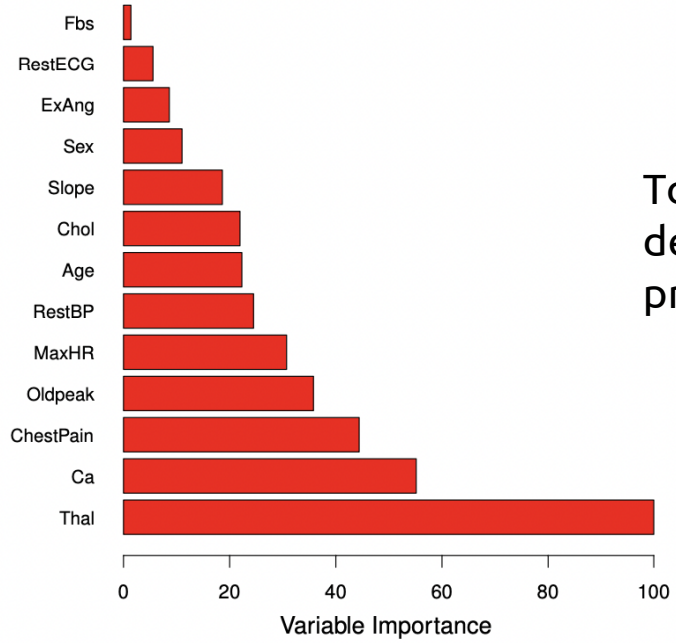
2.1. INTRODUCTION



OOB?

DISCUSSION

Interpretability & Variable Importance in Bagging

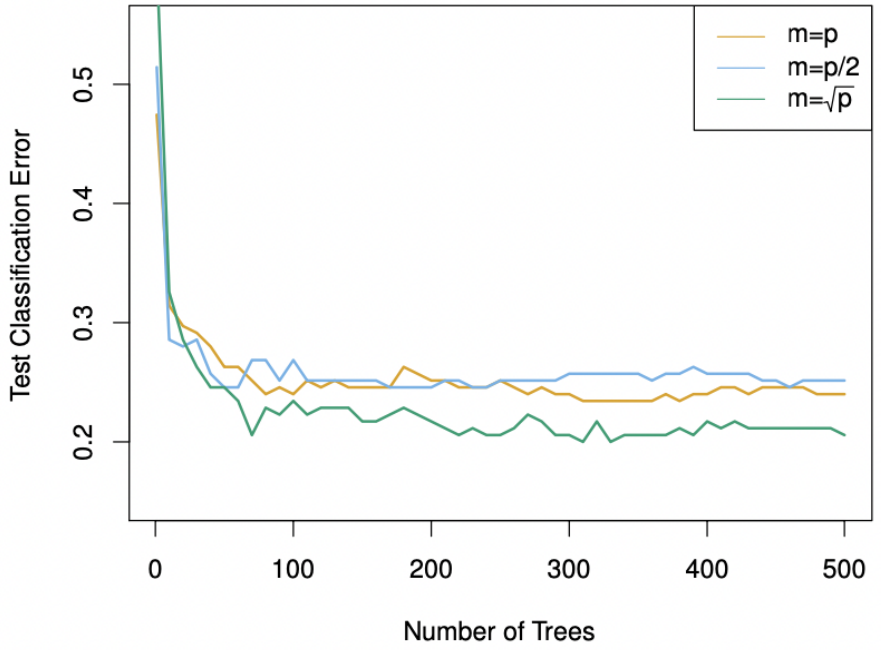


Total amount of RSS decreased over a predictor

2. BAGGING

2.2. RANDOM FORESTS

- An improvement over Bagging by a small tweak that decorrelates the trees.
- Constructing the trees, for each split just a random sample *m* of predictors is chosen rather than the whole set of variables.
- Typically $m \approx \sqrt{p}$
- The idea is to avoid having very similar (correlated) trees in case we have a very strong predictor



AGENDA

1. Decision Trees
   - 1.1. Regression
   - 1.2. Classification
   - 1.3. Pros and Cons
   - 1.4. Trees and KNN

2. Bagging
   - 2.1. Introduction
   - 2.2. Random Forests

3. Boosting
   - 3.1. Introduction
   - 3.2. AdaBoost
   - 3.3. XGBoost
   - 3.4. LightGBM



3. BOOSTING

3.1. INTRODUCTION

- Another technique that can be applied to many statistical methods for regression and classification.
- Here we will see the application in improving decision trees.
- Bagging creates independent trees. *Boosting* works in a similar way but the trees are grown sequentially: each tree uses information from the previous one.
- Each tree is trained with a modified version of the original dataset.

3. BOOSTING

3.1. HOW BOOSTING WORKS

Boosting builds one strong predictor through sequential corrections

1 — Start
Use a simple baseline $F_0(x)$

2 — Fit
Train a shallow tree on current mistakes

3 — Scale
Shrink its contribution with learning rate η

4 — Add
Update the ensemble $F_m = F_{m-1} + \eta h_m$

5 — Repeat
Continue until M trees or early stopping

Final model

$$F^M(x) = F_0(x) + \Sigma_{m=1}^{M} \eta \cdot h_m(x)$$

Sequential

Tree m depends on what trees 1…m−1 still get wrong.

Weak learners

Shallow trees contribute small, complementary rules.

Additive

Predictions accumulate; trees are not averaged independently.

3. BOOSTING

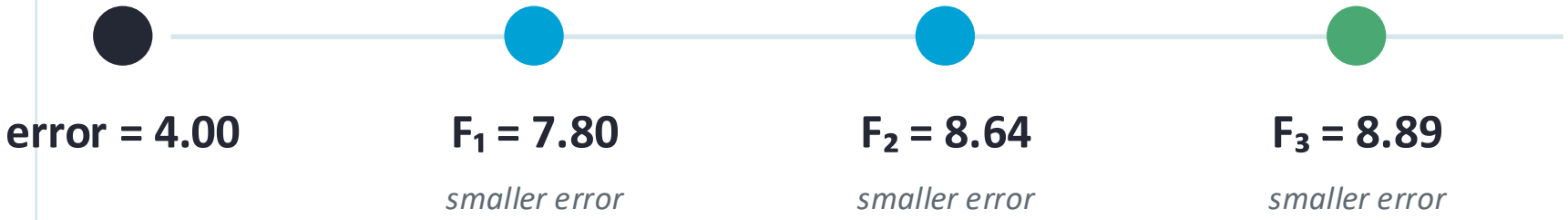
3.1. A REGRESSION EXAMPLE

Each tree learns a smaller correction to the ensemble's current prediction

One observation

Target y = **9.00**

The trees are fitted on all training rows; this path follows how one row is corrected.



With η = 0.7, corrections are deliberately conservative: smaller steps usually need more trees but can generalize better.

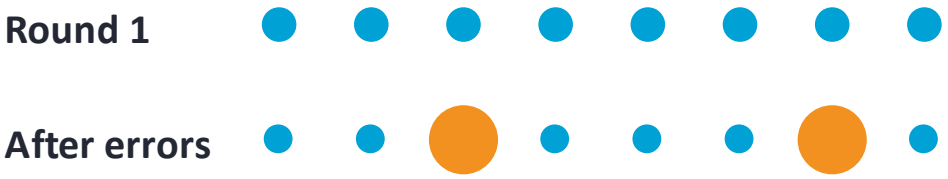
Key interaction: learning rate η ↓ ⇒ number of trees M typically ↑

3. BOOSTING

3.2. ADABOOST

AdaBoost makes repeatedly misclassified observations matter more

Weights evolve after every weak classifier



Misclassified points receive more weight, so the next stump focuses on them.

Output = Weighted vote

$$H(x) = \text{sign}\left[\Sigma_m \alpha_m h_m(x) \right]$$

More accurate weak learners receive a larger vote α_m.

1. **Train** — Fit a weak classifier using the current observation weights.
2. **Measure** — Compute its weighted classification error err_m.
3. **Reweight** — Increase weights on mistakes; decrease weights on correct cases.
4. **Combine** — Add the classifier using a vote based on α_m.

Watch-out: outliers may keep attracting high weight adding noise to the final output.

3. BOOSTING

3.3. GRADIENT BOOSTING

Gradient Boosting trains weak trees to predict the previous ensemble's residuals

Residual with squared-error loss

$$r_{im} = y_i - F_{m-1}(x_i)$$

Each new tree predicts the errors left by the previous ensemble.

1 Evaluate

Compute the current prediction and loss.

2 Compute residuals

Subtract current predictions from observed values.

3 Train a weak tree

Fit a shallow tree $h_m(x)$ to these residuals.

4 Update

$F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$.

For other loss functions

Use pseudo-residuals: the negative loss gradients.

Capacity is controlled by

η · tree depth · number of trees subsampling · early stopping

Core difference

AdaBoost reweights examples; GBM follows gradients of a loss.

3. BOOSTING

3.3. MODERN GBDT IMPLEMENTATIONS

XGBoost, LightGBM and CatBoost optimize the same GBDT idea differently

Model	Core idea	Technical hallmark	Particularly useful when…
XGBoost	Regularized, second-order tree boosting	Sparsity-aware split search; approximate quantiles; scalable systems design	You need a strong, flexible general-purpose tabular baseline.
LightGBM	Histogram-based, leaf-wise GBDT	GOSS and EFB reduce work; leaf-wise growth prioritizes largest gain	Data are large or high-dimensional and speed / memory matter.
CatBoost	Ordered boosting with native categorical handling	Ordered target statistics reduce leakage and prediction shift	Many categorical variables exist and manual encoding is undesirable.

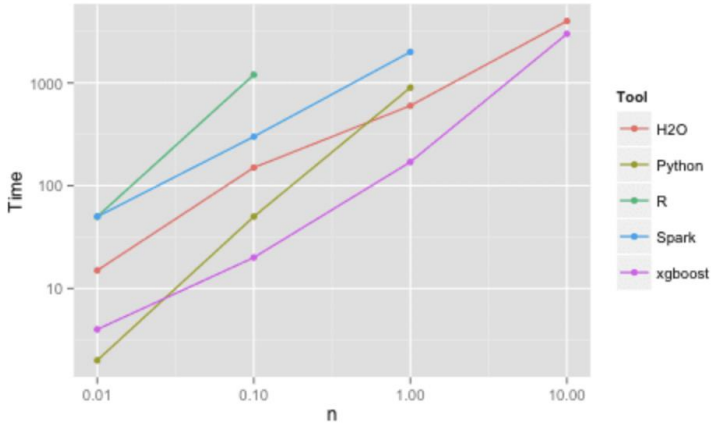
Choose by: validation quality, training cost, categorical handling, and deployment constraints.

3. BOOSTING

3.3. XGBOOST

Execution Speed

Generally, XGBoost is fast. Really fast when compared to other implementations of gradient boosting.

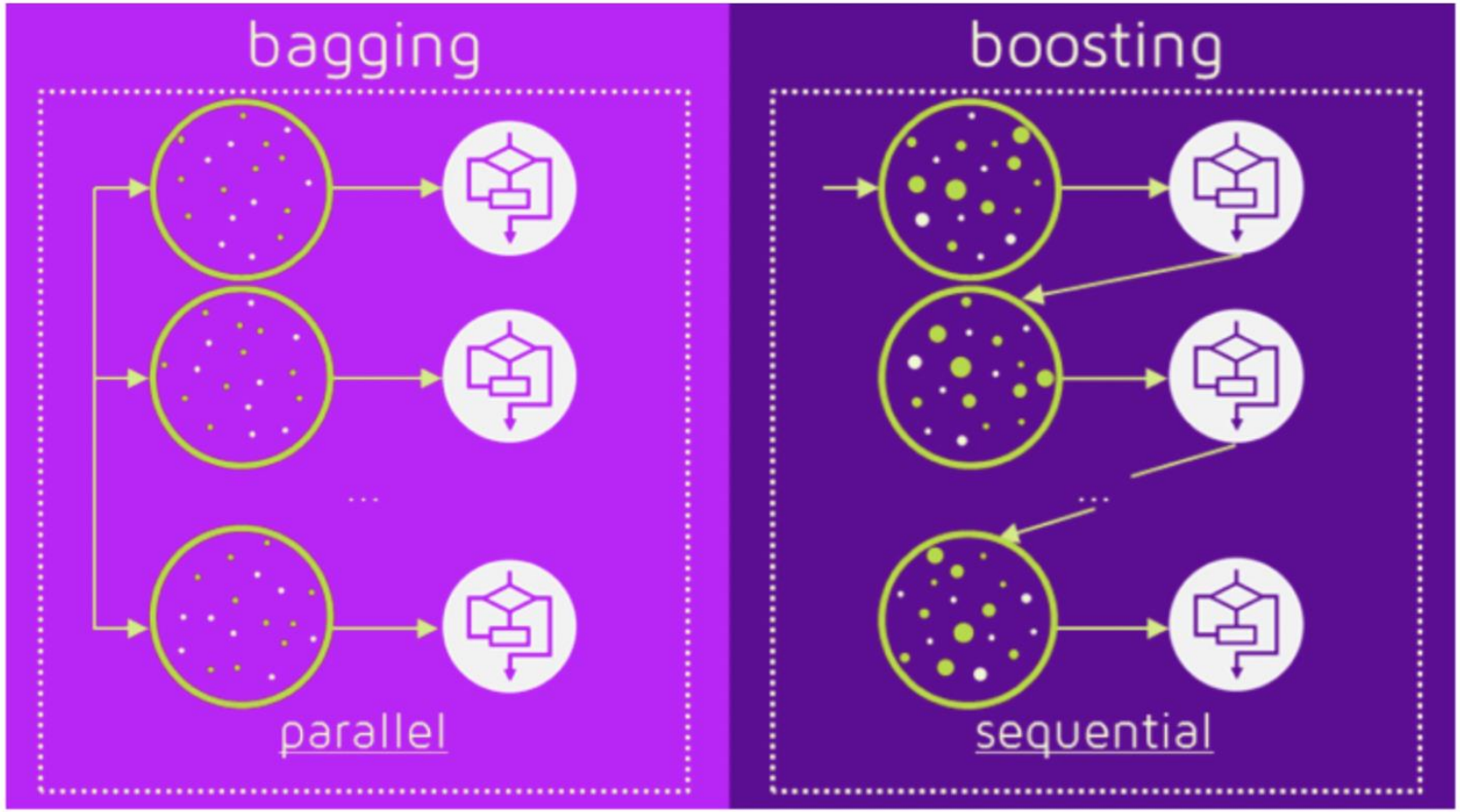


Performance

XGBoost dominates structured or tabular datasets on classification and regression predictive modeling problems.

kaggle

SUMMARY




THANK YOU